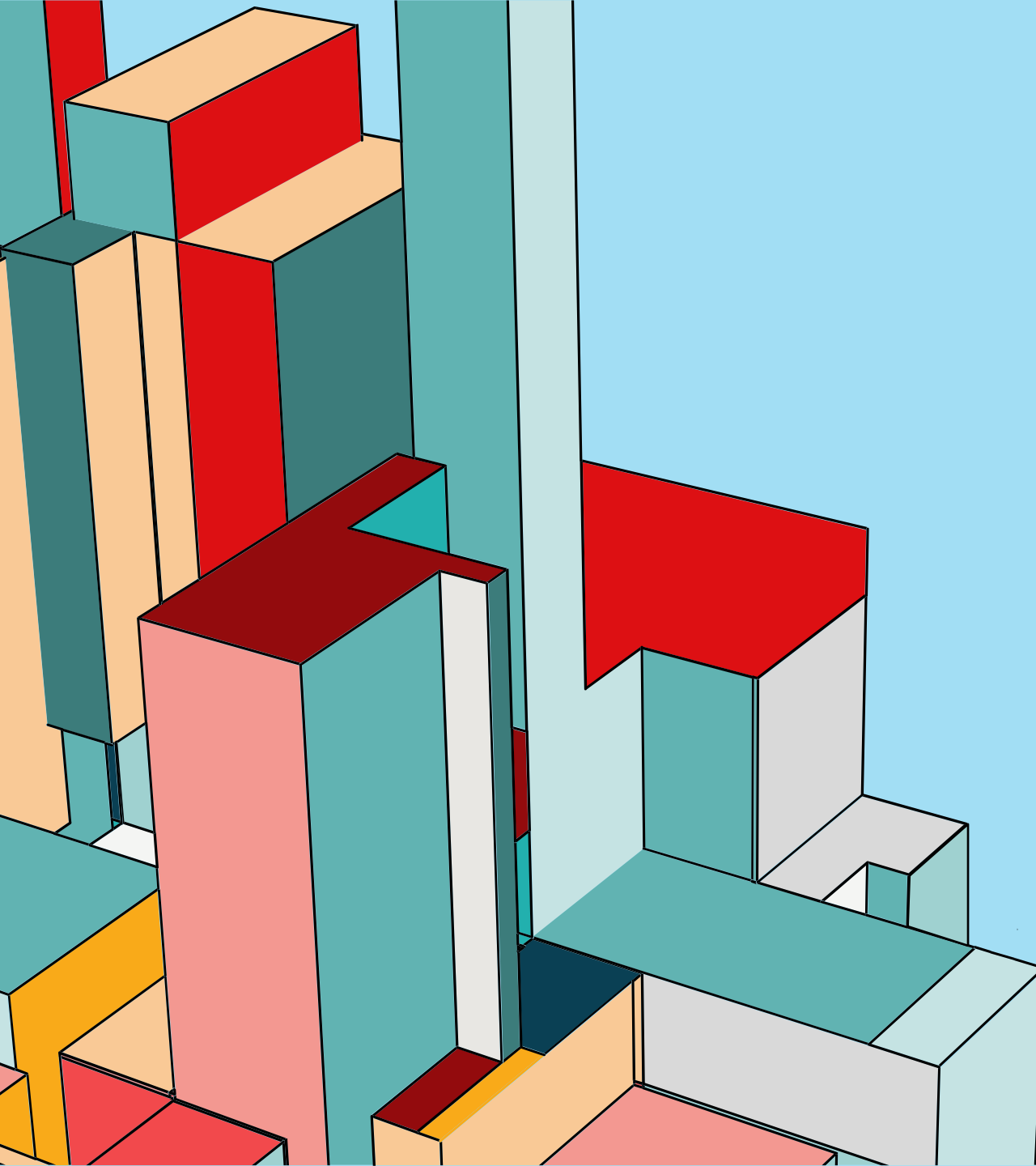




LECTURE 23
LINEAR
PROGRAMMING
AND THE SIMPLEX
ALGORITHM
(+ EXAM 3 REVIEW)

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

FOOD SCIENCE

A company is working on developing a nutrition-replacement bar. It needs to have some minimum amounts of some nutrients and not too much of others.

For simplicity, let's say that our product has just two ingredients:

- Ingredient A (per unit): 5g protein, 2g fat, 4g sodium
- Ingredient B (per unit): 9g protein, 1g fat, 2g sodium

We require at least 9g protein and 3g fat, and a maximum of 10g sodium.

Q: What blend of ingredients A & B should we use?

SETTING UP THE MATH

Let's represent our conditions with some inequalities.

$$5a + 9b \geq 9$$

$$2a + b \geq 3$$

$$4a + 2b \leq 10$$

$$a \geq 0, b \geq 0$$

We can represent each of these graphically. Each one is a line in the plane; the $\geq$ indicates the "high" side of the line; $\leq$ is low.



Q: What shape do these lines enclose? Answer in Submittity.

SETTING UP THE MATH

Let's represent our conditions with some inequalities.

$$5a + 9b \geq 9$$

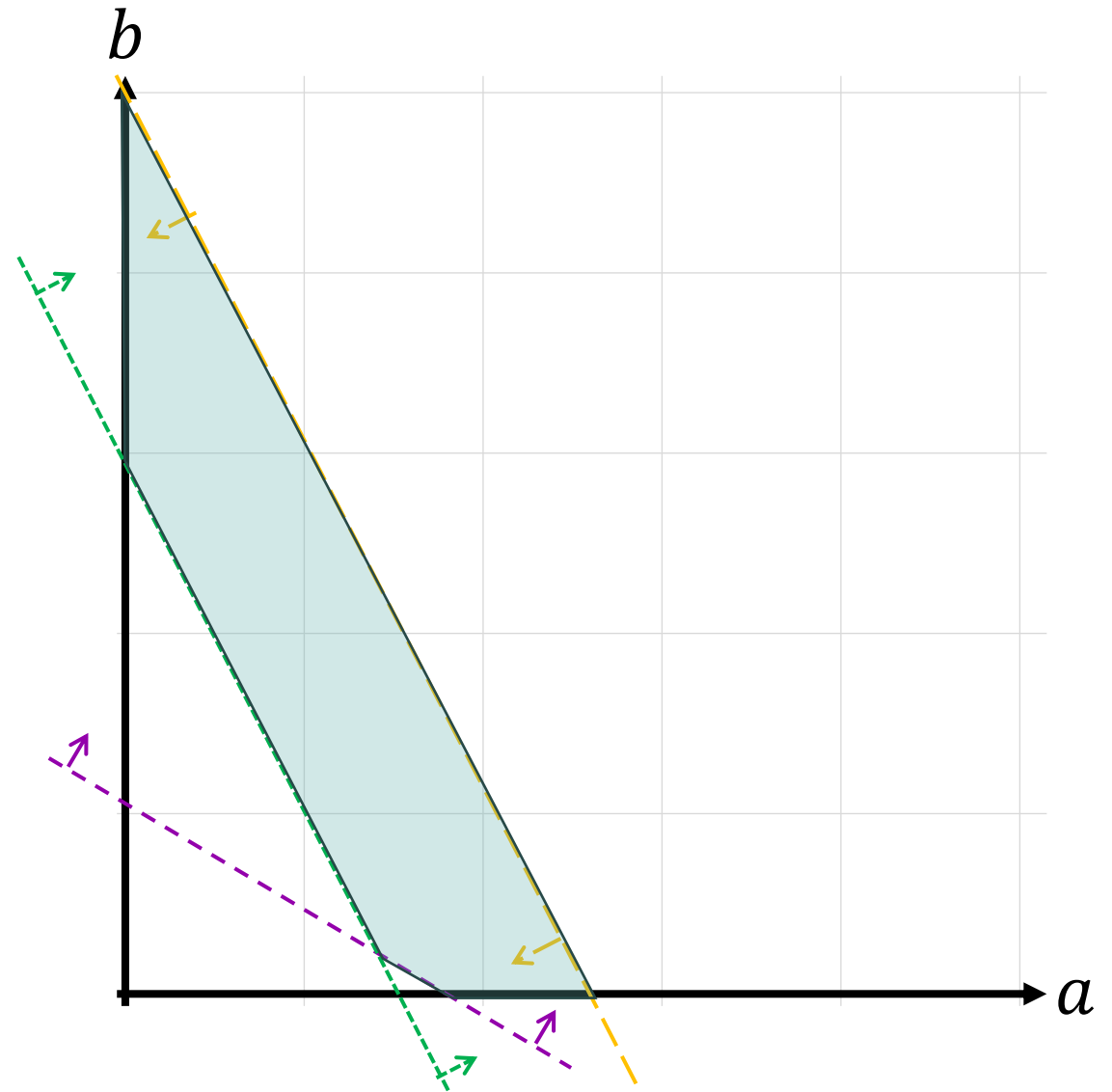
$$2a + b \geq 3$$

$$4a + 2b \leq 10$$

$$a \geq 0, b \geq 0$$

We can represent each of these graphically.

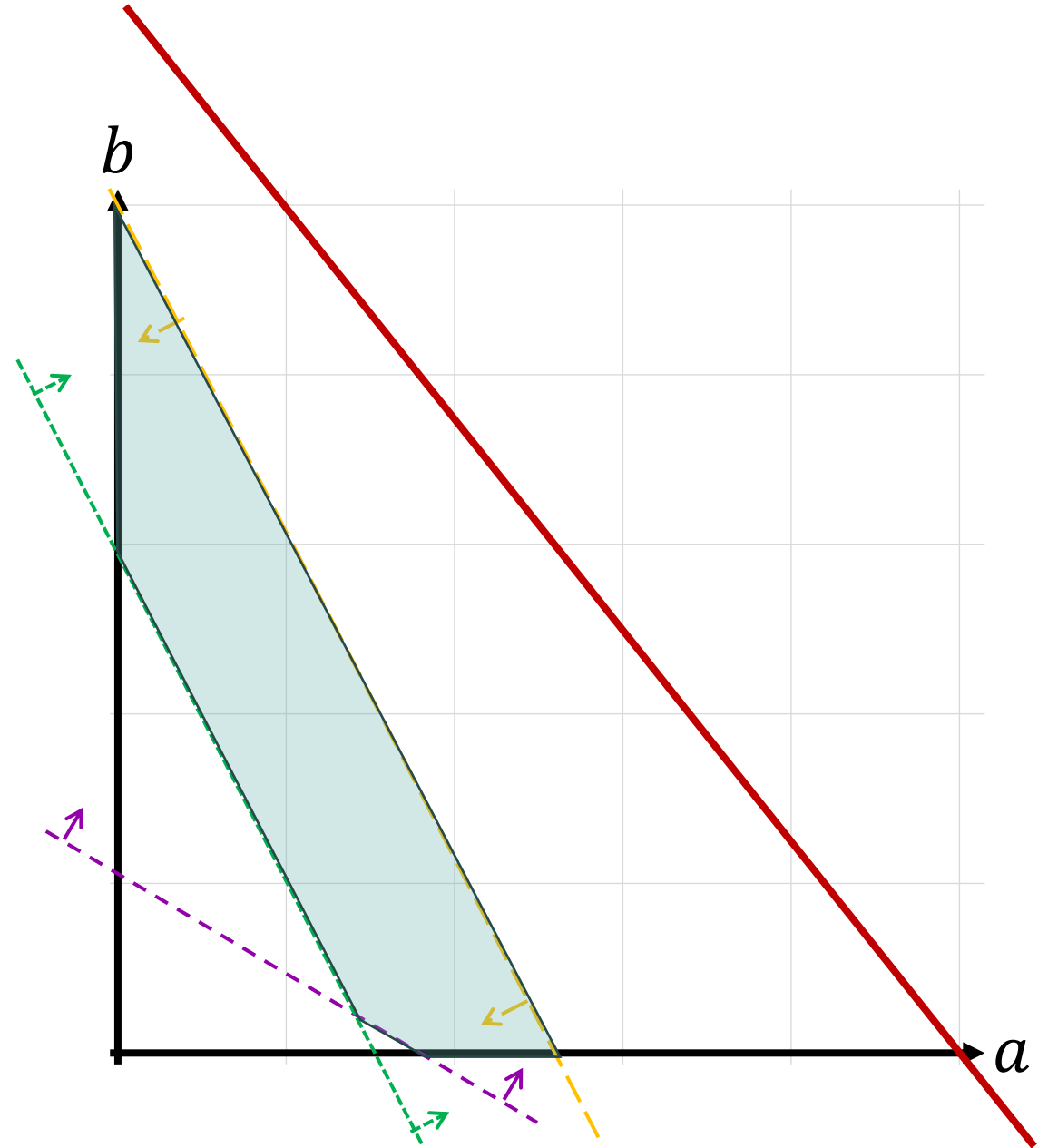
Any point in the shaded region is valid. Which one should we pick?



SETTING UP THE MATH

Most companies will try to minimize cost given the constraints. Let's say that one unit of a costs \$0.50 and one unit of b costs \$0.40.

Then the cost of our bar is $c = 0.5a + 0.4b$. This describes another line – in fact, an infinite number of parallel lines. For example, when $c = 2.5$...

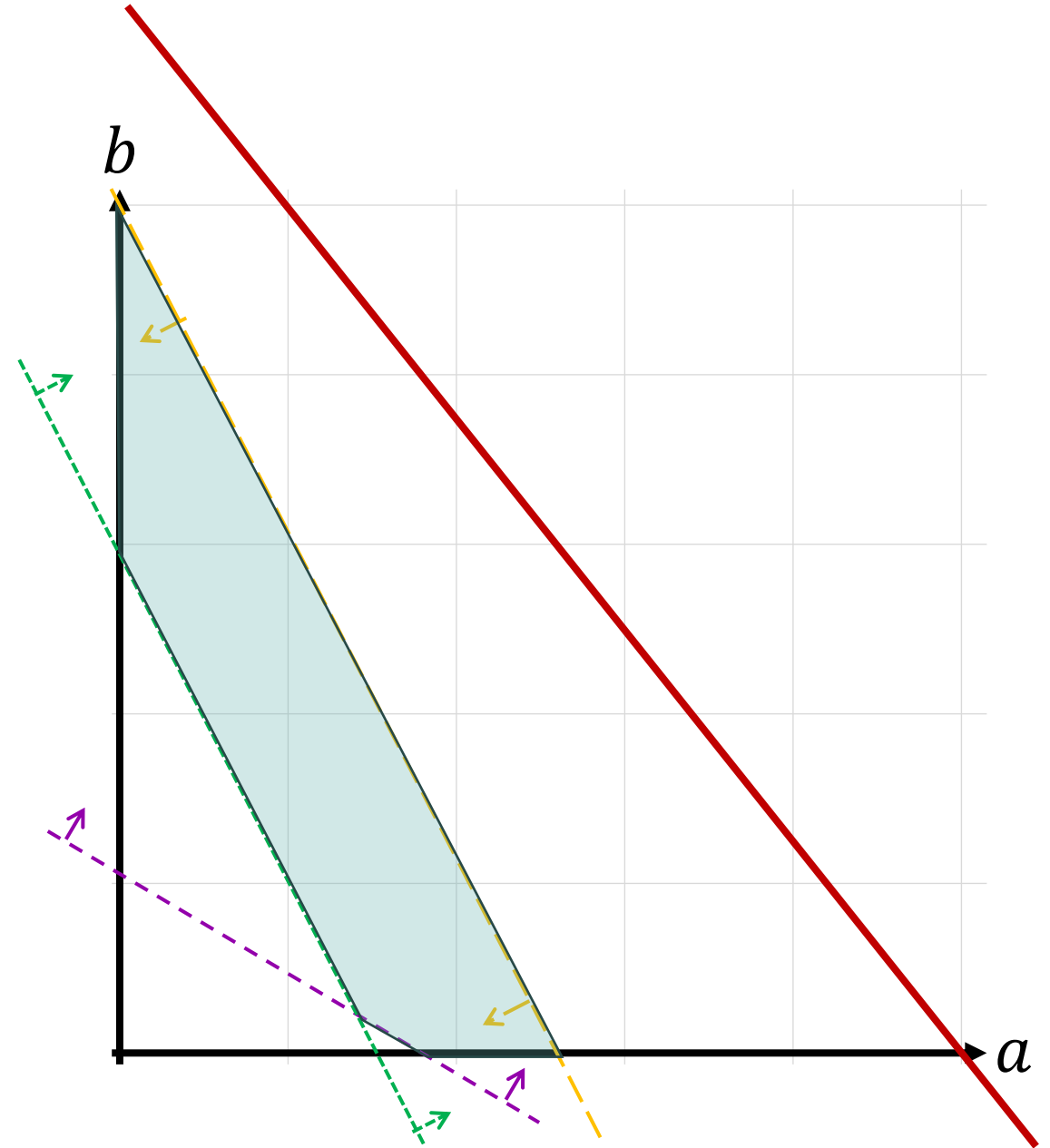


SETTING UP THE MATH

Right now, it doesn't intersect our *feasible region*, but we want to reduce cost anyway.

As we reduce c , that line will move down and to the left.

The optimal solution will be the last point at which the cost line intersects with the feasible region. Where will that be?



SETTING UP THE MATH

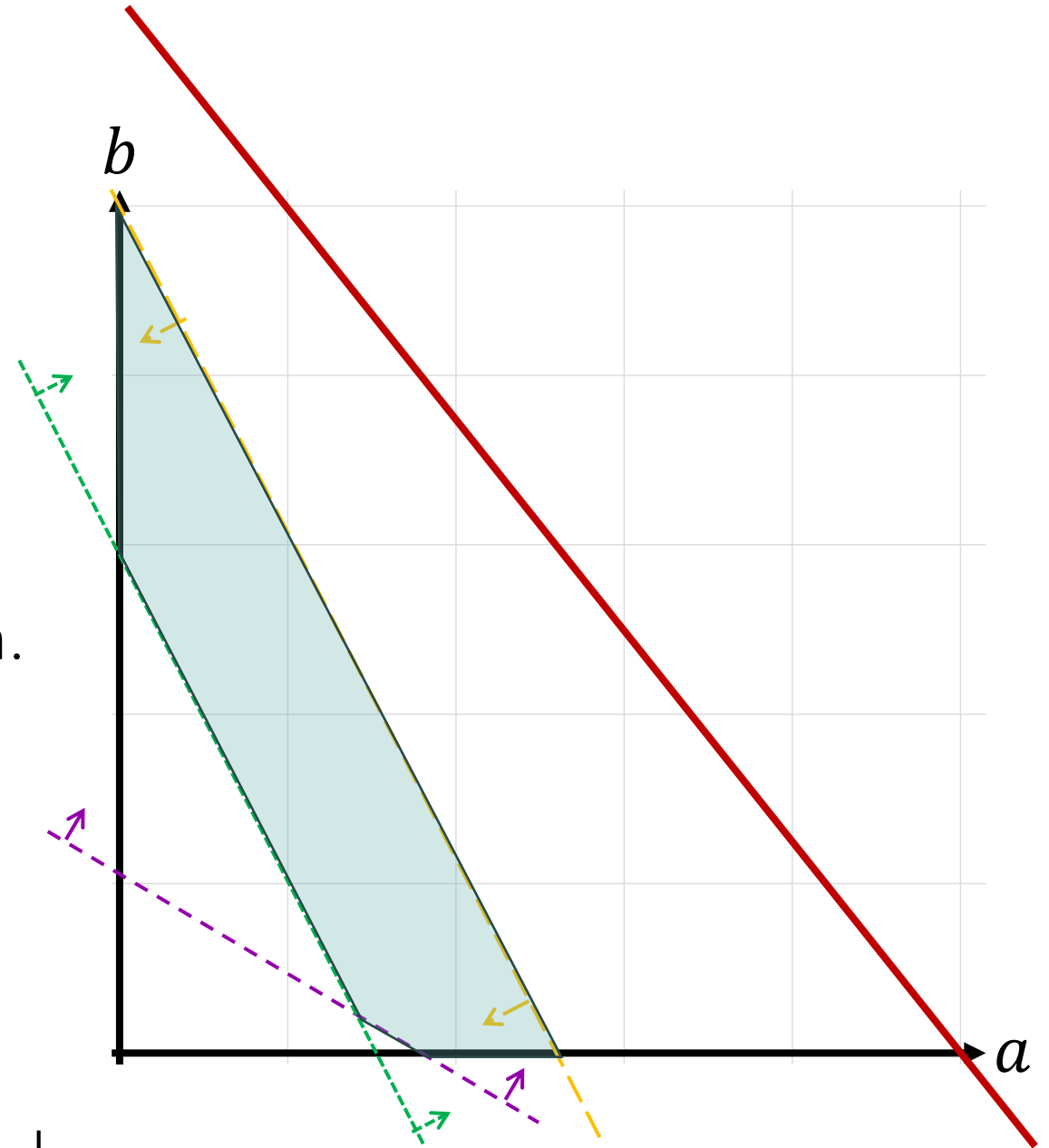
In our example, that will be the corner where the purple and green lines intersect.

The optimal solution ended up at a vertex of our feasible region.

Q: Will that always happen?

Think about how the cost line moves. There are two options:

- The last overlap is a vertex.
- The last overlap is an entire side.



LINEAR PROGRAMMING

Together, the system of constraint inequalities and the cost expression constitute a *linear program*: an optimization problem in which the *objective function* is a linear function of the *decision variables*, and all of the constraints are also linear.

This sort of problem has been studied at least since 1827, when Fourier published a solution technique. But it took over 100 years before Leonid Kantorovich and Wassily Leontief, working independently, started really detailing how useful they could be.

Kantorovich would jointly win the 1975 Nobel Prize in economics with T.C. Koopmans for their work putting many MANY problems in economics on this firm mathematical foundation.

APPLICATIONS OF LINEAR PROGRAMMING

- Supply chain planning: Where to store stuff? Constraints are warehouse capacity, truck weight limits, demand forecasts, etc. Optimize for cost, for delivery time, or some hybrid of the two.
- Power grid optimization: How much power should each power plant put out at different times during the day? Transmission lines have limited capacity, plants have a maximum output, and demand fluctuates all the time. No brownouts!
- Pro sports & salary caps: Need a certain number of players at each position, roster size is limited. Maximize team quality.
- If some of these feel like network flow problems, you're right! Network flow is a special case of linear programming!

HIGHER DIMENSIONS

Obviously, we can't draw all of these in the plane – with n decision variables, we're working in n -dimensional space.

Nevertheless, our core property holds: There is always a vertex of the feasible region that is an optimal solution.

This begs the question: What does the feasible region “look like” in this higher dimensions? And what is a vertex?

In 3-D, the feasible region is a polyhedron, and the various values for the objective function define planes.

More generally, our feasible region is a *convex polytope* in n dimensions – a region with “flat sides” in which any line segment between two points in the region stays in the region.

HIGHER DIMENSIONS

More generally, our feasible region is a *convex polytope* in n dimensions – a region with “flat sides” in which any line segment between two points in the region stays in the region.

The vertices are simply the points at which n constraints all intersect. No matter how many constraints there are, if there is an optimal solution (the system could be infeasible or unbounded), then precisely n of those constraints will be *tight* – right at the limit of the inequality.

SOLVING A LINEAR PROGRAM

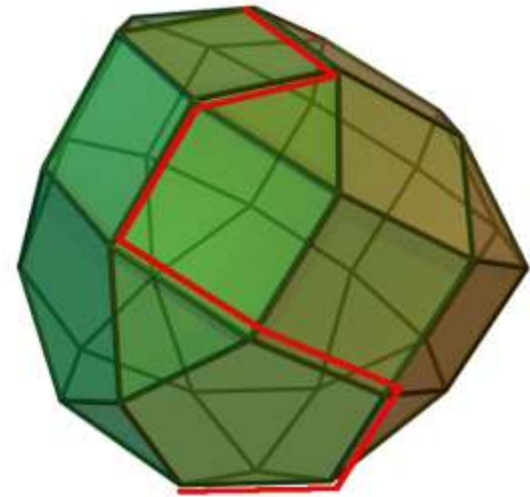
In the mid 1940s, George Dantzig was at (you guessed it!) the RAND Corporation, working on logistics planning for the Air Force using a mechanical desk calculator and wanted to automate the process.

Over the course of a year+, he developed the *simplex algorithm*. This became the heart of his doctoral dissertation – moreover, it was recognized as one of the top 10 algorithms of all time in the Jan/Feb 2000 issue of *Computing in Science & Engineering*.

While the name “simplex” comes from a mathematical term for the shape of the feasible region, the algorithm is, in fact, simple!

THE SIMPLEX ALGORITHM

- Let v be any vertex of the feasible region
- While v has any neighbor v' with a better value for the objective function:
 - Set $v = v'$



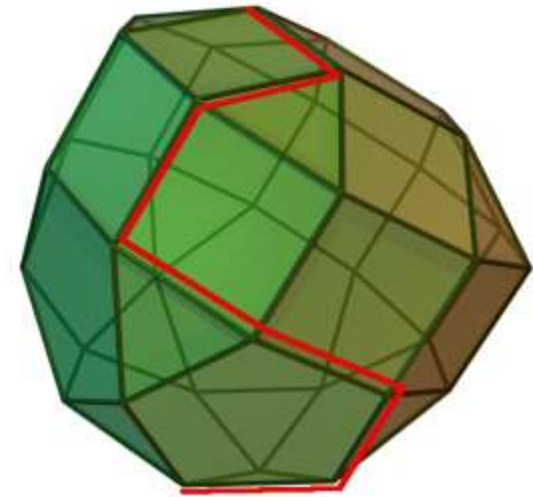
... that's it!

There are a lot of details missing (how do we define the feasible region; how do we find the vertices, etc.), but at its heart, this is all the simplex algorithm does.

NEIGHBORING VERTICES

Possibly the most important thing left out on the previous slide is this: What is a “neighboring” vertex?

The geometric interpretation is easy, but how do we turn that into something we can work with in a computer program?



We know that each vertex is a point at which n constraints are tight.

This actually gives us a very natural definition of “neighbor” – one that corresponds to the geometric intuition.

- Two vertices v_1 and v_2 are neighbors if and only if they have precisely $n - 1$ tight constraints in common.

STANDARD FORM

We next need to pick a starting vertex. The way the simplex algorithm handles this is by ensuring that every LP is in *standard form*. The tools of linear algebra give us some nicely compact notation for this:

Find a vector $\vec{x}$ that maximizes $\vec{c}^T \vec{x}$, subject to $A\vec{x} \leq \vec{b}$ and $\vec{x} \geq \vec{0}$

Here, $\vec{x}$ contains all of the decision variables, $\vec{c}$ are the coefficients in the objective function, and A and $\vec{b}$ contain the constraints.

CHECKING FOR OPTIMALITY

Find a vector $\vec{x}$ that maximizes $\vec{c}^T \vec{x}$, subject to $A\vec{x} \leq \vec{b}$ and $\vec{x} \geq \vec{0}$

Can you find an obvious vertex now?

The origin will always be a vertex of the feasible region of an LP in this form (as long as the feasible region is non-empty)!

It's clearly the intersection of the n constraints represented by $\vec{x} \geq \vec{0}$.

Next up: How do we know if the origin is the optimal vertex?

This one's also easy! Just check the signs of the components of $\vec{c}$ - the origin is optimal if and only if they're all negative!

The constraints say we can only increase the decision variables from here. If the coefficients are all negative, this will reduce the objective.

FINDING A NEIGHBOR

Find a vector $\vec{x}$ that maximizes $\vec{c}^T \vec{x}$, subject to $A\vec{x} \leq \vec{b}$ and $\vec{x} \geq \vec{0}$

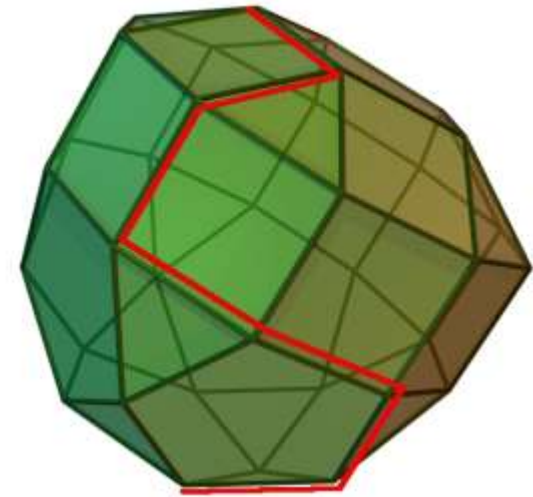
On the other hand, if any c_i is positive, then increasing x_i results in an increase in the objective, so the origin is not optimal.

Further, this points the way to finding a neighboring vertex – we loosen the constraint $x_i \geq 0$, and keep increasing x_i until we bump up against a different constraint.

This must be a new vertex, since it is tight on $n - 1 + 1 = n$ constraints. Furthermore, it shares $n - 1$ constraints with the origin, so it is a neighbor.

A LEVER AND A PLACE TO STAND...

- Let v be any vertex of the feasible region
- While v has any neighbor v' with a better value for the objective function:
 - Set $v = v'$



Ok, we have one iteration covered. How do we tell if this new vertex is optimal, and how do we find the next neighbor?

Simple! Just move the axes so that the new vertex is once again the origin. (This involves updating coefficients and constraints, but it's not hard – it can be done in $O(mn)$ time.)

THE RUNTIME OF SIMPLEX

Given an LP with n decision variables and m constraints, how many vertices might the feasible region have?

Each vertex is tight on n constraints: there are $\binom{m}{n}$ possible sets of size n drawn from a pool of size m .

... but $\binom{m}{n}$ is exponential in n ...

Simplex is an exponential-time algorithm! Why are we even talking about it?

In practice, the exponential cases just... don't come up. On any realistic problem, it runs fast.

OTHER APPROACHES TO LP

In 1979, Leonid Khachiyan published the *ellipsoid algorithm* – one that runs in polynomial time for all cases.

Paradoxically, while it is efficient in theory, it proved much slower than simplex in practice – the ability to handle all of the weird cases slowed it down for the cases that happen in real life.

In 1984, Narendra Karmarkar developed the first *interior-point method* for solving LP. It was also a polynomial-time algorithm, and it was competitive with simplex in practice.

This actually touched off a bit of a competition, with the result that we now have very fast code to run both algorithms!

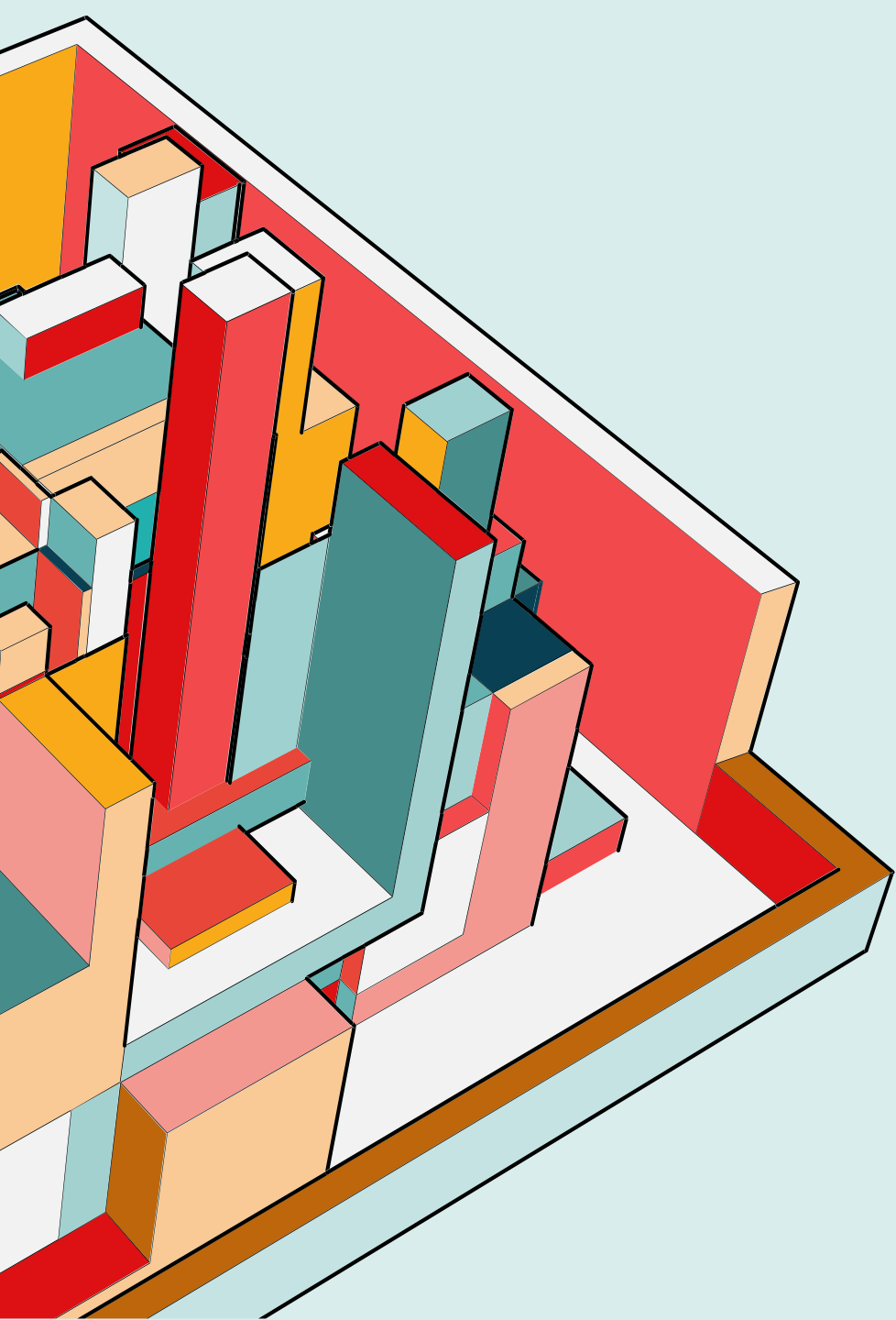
A FINAL WORD

Linear programming is a fundamental algorithm in computer science – doubly so because it is fairly easy to show that literally any polynomial-time algorithm can be transformed into an LP.

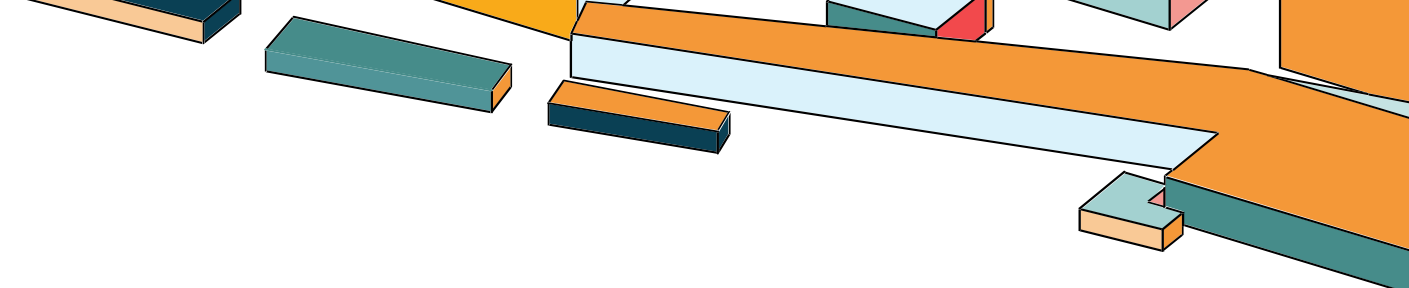
A key aspect of the problem that makes it tractable is that we allow our decision variables to be continuous.

There are many problems where we would prefer to insist that the decision variable only take on integer variables. This type of problem would then be an *integer linear program* or ILP.

Unlike LP, ILP is a hard problem – it's NP-complete!



EXAM 3 REVIEW

An abstract graphic in the top right corner consisting of several 3D rectangular blocks in orange, teal, and light blue, arranged in a staggered, overlapping fashion.

TOPICS

- Dynamic Programming
 - Network Flow
 - Stable Matching
- 
- A solid orange horizontal bar located at the bottom left corner of the slide.

DYNAMIC PROGRAMMING – OUTLINE

To build a dynamic programming algorithm:

1. Characterize the recursive substructure of the problem.
2. Write a recurrence relation that specifies the optimal value of a larger problem in terms of the optimal values of smaller problems.
3. Compute **and record** the values (and path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.
4. Reconstruct the optimal solution from the recorded information.

EXAMPLE: THE ROD-CUTTING PROBLEM

Quick reminder: We have a rod of length n . We may cut it into any number of integer lengths we want; the prices that rods of each length will sell for are given to us as input. We want to find the division of the rod that will maximize revenue.

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

1. Characterize the recursive substructure of the problem.

We can cut off and sell a piece of any size, including none at all. Each time we do so, we have a smaller rod whose revenue we want to maximize. We will have to check all possible cuts.

$OPT(i)$ will be the revenue produced by the best possible division of a rod of length i .

EXAMPLE: THE ROD-CUTTING PROBLEM

Quick reminder: We have a rod of length n . We may cut it into any number of integer lengths we want; the prices that rods of each length will sell for are given to us as input. We want to find the division of the rod that will maximize revenue.

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

2. Write the recurrence relation.

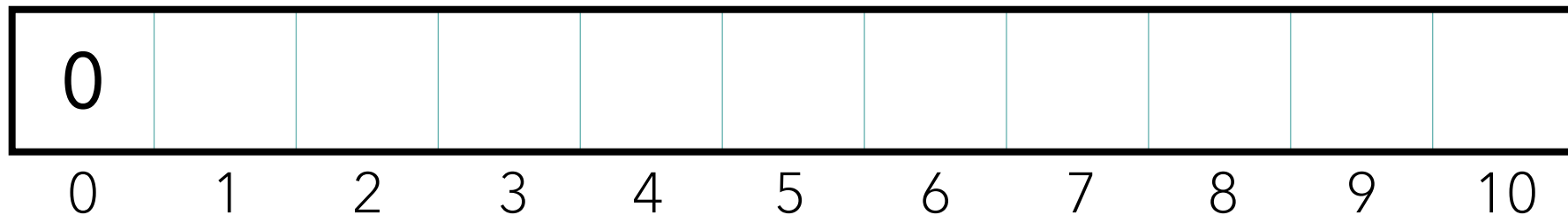
$$OPT(i) = \max_{1 \leq j \leq i} (p_j + OPT(i - j)); OPT(0) = 0$$

EXAMPLE: THE ROD-CUTTING PROBLEM

Quick reminder: We have a rod of length n . We may cut it into any number of integer lengths we want; the prices that rods of each length will sell for are given to us as input. We want to find the division of the rod that will maximize revenue.

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

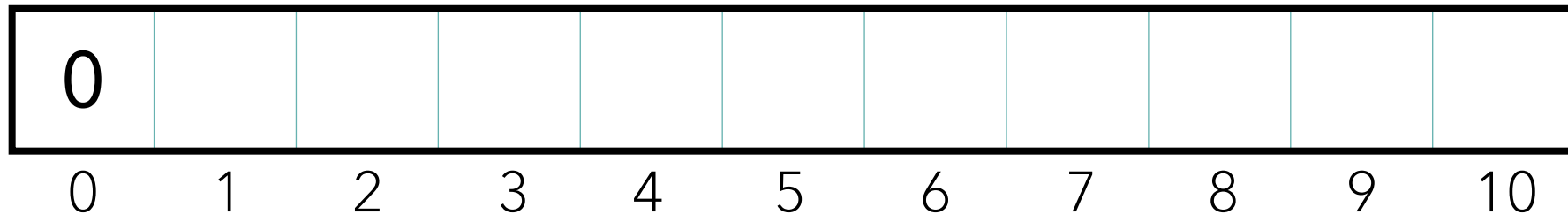
3. Compute and record the values (and possibly path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.



EXAMPLE: THE ROD-CUTTING PROBLEM

3. Compute and record the values (and possibly path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30



$$OPT(i) = \max_{1 \leq j \leq i} (p_j + OPT(i - j)); OPT(0) = 0$$

What is $OPT(10)$? Answer now in Submittity.



EXAMPLE: THE ROD-CUTTING PROBLEM

4. Reconstruct the optimal solution from the recorded information.

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

0	1	5	8	10	13	17	18	22	25	30
0	1	2	3	4	5	6	7	8	9	10

Pretty trivial here: $OPT(10) = p_{10}$, and so we're immediately done.

In general, this involves, in reverse order, checking each entry against a previous one (or more) to see which elements got used.

DYNAMIC PROGRAMMING VARIATIONS

- Look at one previous entry or several? $O(1)$ or $O(n)$ or worse?
- 1-D or 2-D table?
 - If 2-D table, you must specify the order in which the table is going to be filled in.
- Tree structures
 - Can result in 1-D or 2-D table; often triangular.
- Memoization
 - More overhead, but great when we don't need all subproblems

FLOYD-WARSHALL

All-paths shortest path via dynamic programming

Order vertices, then each round allow one new vertex to be an intermediate vertex in the paths (so path lengths will increase from 1 to n)

Results in a 2-D table of distances

NETWORK FLOW

Single source, single sink

Flows must obey capacity constraints and conservation of "stuff"

Ford-Fulkerson procedure finds max-flow and min-cut

- Residual graph with reversed edges

- F-F needs a selection method for picking the next augmenting path or it can get bogged down; Edmonds-Karp uses the path with the fewest links.

VARIATIONS & APPLICATIONS

Circulations: Multiple sources, multiple sinks

Minimum capacities

Modeling situations using a network flow

Often scheduling or similar “assignment” problems

STABLE MATCHING

Bipartite matching with preference rankings

Stability == no one can improve their position unilaterally

Gale-Shapley algorithm: proposers & accepters

Best possible result for proposers; worst possible result for accepters

Proofs here rely heavily on contradiction: "Assume X. Then we're supposed to have a stable matching. Whoops - it's not so stable after all! X is false."

FEEDBACK FORM

Please comment on today's class.



SETTING UP THE MATH

Let's represent our conditions with some inequalities.

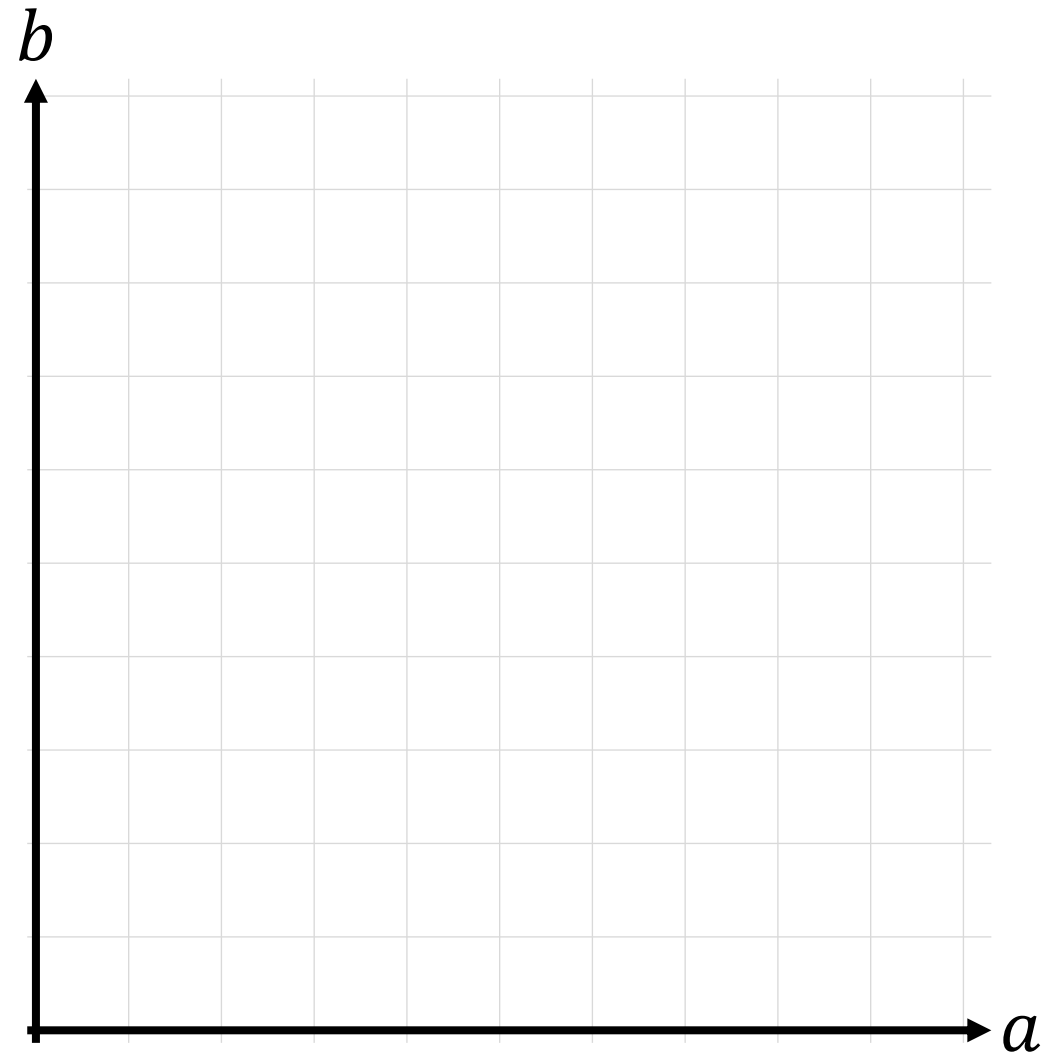
$$5a + 7b \geq 8$$

$$4a + 2b \geq 15$$

$$2a + b \geq 3$$

$$a \geq 0, b \geq 0$$

We can represent each of these graphically.



SETTING UP THE MATH

Let's represent our conditions with some inequalities.

$$5a + 7b \geq 8$$

$$4a + 2b \geq 15$$

$$2a + b \geq 3$$

$$a \geq 0, b \geq 0$$

We can represent each of these graphically.

